

CSED 232 Object-Oriented Programming (fall 2022)

Programming Assignment #1

-Programming Practice-

Da Vinci Code

Due data: 2022년 10월 5일 수요일 23:59

담당조교: 김승욱 (wookiekim@postech.ac.kr)

첫번째 Assignment는 학생들이 C 프로그래밍에서 배웠던 것을 remind하고 C++ 프로그래밍의 기초적인 실습, 그리고 컴파일 환경 등을 세팅해서 컴파일 해 볼 수 있게 하는게 목적입니다.

● 제출 방식

- '학번'(ex: 20202121)으로 된 폴더 안에 코드 파일들 (*.cpp, *.h)과 제출 보고서를 포함하도록 한다
- 제출 보고서는 보고서 양식에 따라 필요한 내용을 중심으로 간결하게 작성하며, pdf 파일로 제출한다. 파일 이름은 '학번.pdf' 로 한다. (코드를 통째로 첨부 X. 꼭 필요한 내용만 넣을 것)
- **명예서약(Honor Code):** 보고서 표지 및 작성한 코드 main마다 주석으로 다음 내용을 포함한다. "나는 이 프로그래밍 과제를 다른 사람의 부적절한 도움 없이 완수하였습니다." 없는 경우 과제를 제출하지 않은 것으로 처리한다.
- '학번' 폴더를 '학번.zip'(20202064.zip) 파일로 묶어서 제출한다.
- 소스 코드(*.h, *.cpp), Makefile, 보고서 외에 불필요한 파일(*.obj, *.o, *.exe, *.out)들은 zip 파일에 포함시키지 않는다.
→ **Makefile 제출은 필수가 아니며**, 사용한 컴파일러 (운영체제)에 따라 선택적으로 제출할 수 있다.
예시로, Visual Studio를 통해 구현했으면 소스코드만 제출해도 괜찮다.
- **이번 프로그래밍 과제의 코드 파일 구조는 자유입니다. 하나의 main.cpp 파일로 제출해도 되고, 필요에 따라 여러 .h 파일 및 .cpp 파일을 정의해도 좋습니다.**

● 제출 방법

- PLMS(<https://plms.postech.ac.kr/>)의 객체지향 프로그래밍 과제메뉴에서 Assignment1 클릭
- '학번.zip' 압축파일을 첨부파일로 기간 내에 제출한다.
- **제출 기한이 지나면 하루마다 얻은 총점의 20% 감점**
- **1일 이내 지연 20% 감점, 2일 이내 지연 40% 감점, 5일 이상 지연 0점**

● 채점 방식

1) 프로그램 기능 (전체 점수 60%)

- 프로그램이 요구 사항을 모두 만족하면서 올바르게 실행되는가?

2) 프로그램 설계 및 구현 (전체 점수 30%)

- 설계된 내용이 C++를 이용하여 적절히 구현되었는가?
- 프로그램 설계 시 기능별로 모듈화가 잘 이루어졌는가?
- 프로그램 기능과 구조를 보고서에 잘 기술하였는가?

3) 프로그램의 가독성 (전체 점수 10%)

- 프로그램이 읽기 쉽고 이해하기 쉽게 작성되었는가?
- 변수 명과 함수 명이 무엇을 의미하는지 알아보기 쉬운가?
- 프로그램의 소스코드를 이해하기 쉽도록 주석을 잘 붙였는가?

4) 추가 점수 (최대 전체 점수 10%)

- 기본 요구 사항 외의 추가 점수 기능을 구현, 보고서에 잘 정리하였을 경우 최대 전체 점수의 10%를 추가점수를 부여한다.
- 추가 점수 항목을 구현 및 작성하였을 경우 이를 보고서의 토론 및 개선 항목에 **빨강게** 표시한다.

5) 감점 주의 사항

- 동적 할당 메모리 사용시 할당 해제를 제대로 하지 않을 경우 해당 문제 점수 10% 감점
- 문제마다 제약사항이 존재. 제약사항들을 잘 확인할 것. 지키지 않을 시 해당 문제 0점.
- 출력 양식을 지키지 않을 시 해당 문제 0점.
- 보고서와 각 코드 main 위에 (주석으로) Honor code를 명시해야 한다. 없을 시 과제 전체점수 0점.
- Cheating시 F 학점은 물론 대학교 차원의 징계가 있을 것. 모든 코드는 인터넷 상의 코드와 대조해 볼 것이니 인터넷 상의 코드를 참고할 경우 이해한 후 본인의 방식으로 새로 짤 것.
- 참고한 자료들 및 사이트는 보고서에 reference 형태로 적어야 한다.
- 제출 양식(폴더 명, 파일 명, 파일 저장 위치 등)을 지키지 않을 시 과제 전체 점수 10 % 감점

● 기타

- STL 사용 불가.
- 잘못된 입력은 주어지지 않음. 예외처리를 할 필요는 없으나 항상 예외처리를 고민하시는 것을 권장.
- **과제에 대한 문의 사항이 있다면 Assign1 담당 조교 이메일로 질문.**

과제 설명: Da Vinci Code

(목적)

이번 과제를 통하여 C++에서의 조건문, 반복문, 사용자 정의 함수 및 라이브러리 함수 사용법을 익힌다. 또한, C언어에서 배웠던 구조체, 포인터, 링크드 리스트 등의 개념을 C++로 구현해보는 연습을 한다.

(주의사항)

- 문제의 출력 형식은 채점을 위해 아래의 실행 예시와 최대한 비슷하게 작성해 주세요.
- 룰이 원래의 다빈치 코드 게임 룰과 살짝 다르니 유의해 주세요.
- 마지막 페이지의 '3. 기타사항'을 먼저 확인 후, 설계 및 구현을 하도록 하세요.

(설명)

본 과제는 유명한 보드게임인 다빈치 코드를 C++로 구현하는 문제이다. 다빈치 코드는 숫자가 쓰여 있는 흑백의 타일을 나눠갖고, 다른 사람의 타일에 쓰인 숫자를 보지 않고 맞추는 추리게임이다. 원래의 룰에 대한 설명은 [여기](#)서 확인해볼 수 있다.

우리가 구현할 다빈치 코드도 전반적으로 원래 룰과 비슷하게 진행되지만, 다음과 같은 차이가 있다.

1. 조커 타일은 존재하지 않는다. (구현시 추가점수 8%)
2. 원래는 상대방의 타일을 맞히는데 성공한 경우 계속 추리를 이어가거나 턴을 넘길 수 있는데, 본 과제에서는 상대방의 타일을 맞히는데 성공한다면 턴을 넘길 수 없으며, 추리를 계속해야 한다. (턴을 선택적으로 멈출 수 있도록 하면 추가점수 2%)

본 과제에서 구현할 게임의 방법은 다음과 같다. 0부터 11의 숫자가 적힌 흑/백색의 타일 총 24개가 존재한다. (조커 타일을 구현 시 추가 점수 8%) 컴퓨터는 랜덤으로 4개의 타일을 가져오게 되며, 플레이어는 원하는 색의 타일을 4개 가져온다. 가져온 타일을 왼쪽부터 오른쪽까지 오름차순으로 정렬한다. 만일 숫자가 같은 타일이 둘 다 있을 경우 (예: 흑색 4, 백색 4), 흑색 타일이 백색 타일의 왼쪽에 오게 한다. (조커는 원하는 위치에 자유롭게 둘 수 있다.)

컴퓨터가 첫 턴을 가져가게 되며, 이후 플레이어와 번갈아가며 턴을 가지게 된다.

한 턴의 진행) 매 턴마다 남아있는 타일 중 하나를 가져온다. 이후 상대방의 숫자가 보이지 않는 타일 중의 하나를 골라 해당 타일의 숫자를 맞혀야 한다. 숫자를 맞히는데 성공하면 그 타일은 숫자가 보이도록 공개된다. 추리는 숫자를 맞히는데 실패할 때까지 계속된다 (상대방의 타일을 맞히는 데 성공할 경우 턴을 멈출 수 있도록 구현하면 추가점수 2%). 추리를 실패할 경우 해당 턴에 가져온 타일을 공개하고, 상대방의 턴으로 넘어간다.

- **컴퓨터의 턴**) 남아있는 타일 중 하나를 랜덤으로 가져온다. 이후 상대방의 숫자가 보이지 않는 타일 중 하나를 랜덤으로 추리 대상으로 선택한다. 해당 타일의 색상을 가진 모든 타일 중, 자신에 눈에 보이는 타일의 숫자를 제외한 0~11의 숫자 중 하나를 랜덤으로 추리한다. 이후 실행 예시를 통해 더욱 설명하도록 하겠다.

- **플레이어의 턴**) 원하는 색상의 타일을 가져온다. 이후 상대방의 숫자가 보이지 않는 타일 중 하나를 추리 대상으로 지정 선택한다. 자신이 원하는 숫자로 추리를 한다.

승리 조건) 타일이 모두 공개되면 패배한다.

1. 초기 선택 메뉴

본 과제에서 구현하는 다빈치 코드는 텍스트 기반의 UI를 통해 동작한다. 시작 메뉴를 출력해주고, 메뉴에서 사용자의 선택을 입력 받아 게임 설명 출력, 게임 진행, 또는 게임 종료를 할 수 있다.

```
[ 다빈치 코드 ]
=====
1.  게임 설명
2.  게임 시작
3.  종료
=====
선택:
_
```

그림 1. 다빈치 코드 메인 메뉴 예시

- 사용자가 1을 입력하면, 아래처럼 게임 설명을 출력한 후, 다시 초기 선택 메뉴를 출력하고 사용자 입력을 받을 준비를 한다.

```
=====  게임 설명  =====
다빈치코드는 숫자가 쓰여 있는 타일을 나눠갖고, 다른 사람의 타일에 쓰인 숫자를 맞추는 게임이다.
각 타일은 숫자 0~11로 구성되며, 흰색 또는 검정색을 띤다. (조커는 없다고 가정한다.)
이번 프로그래밍 과제에서는 사용자와 컴퓨터간의 게임, 즉 2인 플레이를 가정한다.
각 플레이어는 게임을 시작할 때 원하는 4개의 타일을 숫자를 보지 않고 가져간다.
자신의 패에 있는 타일은 가장 작은 수 부터 가장 큰 수까지 순서대로 배열해야 한다.
검은색과 흰색이 같은 숫자일 경우 검은색을 먼저 오도록 놓는다.
컴퓨터가 첫 턴을 가져가는 것으로 한다.
자신의 차례에는 뒤집어진 타일을 하나 가져와 규칙대로 자기 패에 끼워넣는다.
이때 가져온 타일이 조커타일이라면 자신이 원하는 위치에 집어 넣을 수 있다.
상대 플레이어의 타일 하나를 지목해 숫자를 맞춰야 한다.
맞힌 숫자가 맞다면 상대는 그 타일을 보이도록 넘혀 공개해야 한다.
맞혔을 경우, 다른 타일의 숫자를 계속해서 맞춰본다. (또는 차례를 넘길 수 있다.)
틀렸을 경우, 그 차례에 가져갔던 타일을 넘혀 숫자를 공개해야 한다.
숫자가 모두 드러난 사람은 패한다.
=====
[ 다빈치 코드 ]
=====
1.  게임 설명
2.  게임 시작
3.  종료
=====
선택:
_
```

그림 2. 다빈치 코드 게임 설명

- 정수 1, 2, 3을 제외한 다른 값이 입력될 때는 다시 입력 받는다. 숫자 이외의 입력은 없다고 가정한다. (숫자에 대한 예외처리만 하면 됨).
- 사용자가 3을 입력하면, 프로그램을 종료한다.

```
[ 다빈치 코드 ]
=====
1.  게임 설명
2.  게임 시작
3.  종료
=====
선택:
4
다시 선택하세요

[ 다빈치 코드 ]
=====
1.  게임 설명
2.  게임 시작
3.  종료
=====
선택:
3
C:\Users\seung\source\repos\Assn1_DaVinciCode\x64\
드: 0개).
이 창을 닫으려면 아무 키나 누르세요..._
```

그림 3. 예외처리 및 게임 종료 예시

- 종료 시 Memory Leak이 없는지 잘 확인해보도록 한다. (Memory Leak의 경우 10% 감점)

2. 게임 시작 (2를 선택시)

사용자가 2를 선택하면, 아래 그림처럼 가져오고 싶은 색상의 타일을 정의할 수 있다. 흑 백 순으로 숫자를 입력하며, 입력된 숫자의 합이 4가 되도록 예외처리를 해줘야 한다.

```
[ 다빈치 코드 ]
=====
1.  게임 설명
2.  게임 시작
3.  종료
=====
선택:
2
가져올 타일의 색 별 개수를 정하세요. (흑 백 순서, 흑+백 = 4):
2 1
가져올 타일의 색 별 개수를 정하세요. (흑 백 순서, 흑+백 = 4):
2 3
가져올 타일의 색 별 개수를 정하세요. (흑 백 순서, 흑+백 = 4):
2 2
```

컴퓨터는 랜덤으로 4개의 타일을 가져오도록 한다. 흑/백을 먼저 랜덤하게 고르고, 이후 흑/백에 남아있는 타일중 하나를 랜덤으로 가져오는 방식을 반복하면 된다.

주의사항) 남아있는 흑색 타일, 남아있는 백색 타일, 컴퓨터의 타일 텍, 플레이어의 타일 텍은 모두 Linked List로 구현해야 한다. 타일을 나타내는 구조체를 정의해야 한다.

컴퓨터가 랜덤으로 타일을 선택하고, 플레이어가 지정한대로 타일을 선택한 이후에는 게임보드를 다음과 같이 출력한다.

```

=====
남은 타일 || □: 7 ■: 9
=====

==컴퓨터 덱==
[1: □??] [2: □??] [3: ■??] [4: □??]

==플레이어 덱==
[1: □4] [2: □6] [3: ■8] [4: ■11]

```

위와 같이 남은 흑색 타일과 백색 타일의 개수를 출력해줘야 한다. 또한, 컴퓨터 덱과 플레이어 덱을 그 밑에 출력해준다. 흑색 타일은 ■, 백색 타일은 □로 나타낸다.

주의사항)

- 컴퓨터의 덱에 있는 타일들은 숫자가 보이지 않기 때문에 물음표 (??) 로 나타내야 한다.
- 타일 왼쪽에 있는 숫자는 위치를 의미한다.
- 상대방에게 보여지지 않은 타일은 대괄호 [] 로, 상대방에게 보여지는 타일은 홑따옴표 <>로 나타낸다.
- 덱에 있는 타일은 오름차순으로 배열되어야 한다. 만일 숫자가 같은 흑 백의 타일이 둘 다 있을 경우, 흑색을 왼쪽에 두도록 한다.

이후 컴퓨터의 턴이 시작된다.

```

=====
남은 타일 || □: 7 ■: 9
=====

==컴퓨터 덱==
[1: □??] [2: □??] [3: ■??] [4: □??]

==플레이어 덱==
[1: □4] [2: □6] [3: ■8] [4: ■11]
컴퓨터의 차례입니다.
컴퓨터가 랜덤 타일을 고르고 있습니다...
컴퓨터가 백색 타일을 골랐습니다.

```

- 컴퓨터의 차례라는 메시지가 출력된다.
- 컴퓨터는 남아있는 타일 중 랜덤한 타일을 골라 온다. 남아있는 타일이 없거나, 남아있는 타일의 색이 하나밖에 없는 경우 등에 대한 처리가 필요하다. 남아있는 타일이 없는 경우, 바로 추리를 진행한다.
- 어떤 색을 골랐는지에 대한 메시지가 출력된다.

```

=====
남은 타일 || □: 7 ■: 8
=====

==컴퓨터 덱==
[1: ■??] [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 덱==
[1: □4] [2: □6] [3: ■8] [4: ■11]

```

- 1번 위치에 흰색 타일이 추가된 것을 알 수 있다.
- 이후 플레이어 덱에 대한 추리를 시작한다.

```

=====
남은 타일 || □: 7 ■: 8
=====

==컴퓨터 덱==
[1: ■??] [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 덱==
[1: □4] [2: □6] [3: ■8] [4: ■11]
컴퓨터가 추리를 시작합니다.
컴퓨터가 3번째 위치의 숫자가 8라고 추리합니다.
정답입니다!
플레이어의 백색 타일 8번이 공개됩니다.

=====
남은 타일 || □: 7 ■: 8
=====

==컴퓨터 덱==
[1: ■??] [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 덱==
[1: □4] [2: □6] <3: ■8> [4: ■11]

```

- '컴퓨터가 추리를 시작합니다' 메시지를 출력한다.
- 컴퓨터가 타일이 공개되지 않은 플레이어 덱의 위치 중 하나를 랜덤으로 고른다.
- 컴퓨터가 고른 타일의 색상에 대해 보이지 않는 숫자 (0~11) 중 하나를 랜덤으로 선택한다.
예) 위 화면에서는 3번째 위치가 8이라고 추리하고 있다. 플레이어의 3번째 위치에는 백색의 8번 타일이 있다. 이 말은 컴퓨터의 눈에는 백색의 8번 타일이 보이지 않는다는 뜻이다 (백색의 8번 타일을 가지고 있지 않으며, 플레이어의 덱에도 백색의 8번 타일이 공개되어 있지 않음.)
- 컴퓨터가 정답을 맞힌 경우: 해당 타일이 공개된다. 3번 위치의 백색 8번 타일이 [] 에서 <>로 변한 것을 알 수 있다. 해당 타일이 컴퓨터에게 공개되었다는 뜻이다. 이후 컴퓨터는 정답을 맞혔으니 추리를 이어간다.

```

=====
남은 타일 || □: 7 ■: 8
=====

==컴퓨터 덱==
[1: ■??] [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 덱==
[1: □4] [2: □6] <3: ■8> [4: ■11]
컴퓨터가 추리를 시작합니다.
컴퓨터가 4번째 위치의 숫자가 7라고 추리합니다.
틀렸습니다!
컴퓨터의 백색 타일 0번이 공개됩니다.

=====
남은 타일 || □: 7 ■: 8
=====

==컴퓨터 덱==
<1: ■0> [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 덱==
[1: □4] [2: □6] <3: ■8> [4: ■11]

```


- **컴퓨터가 틀린 경우:** 컴퓨터가 틀린 경우 해당 턴에 들고 온 타일이 공개된다. 만일 남아있는 타일이 없어서 해당 턴에 들고 온 타일이 없다면, 가장 낮은(왼쪽) 위치에 있는 타일 중 공개되지 않은 타일을 공개한다. 1번 위치의 타일이 []에서 <>로 바뀌며, 물음표 (??) 가 숫자 (0)으로 변한 것을 알 수 있다. 턴이 플레이어에게 넘어온다.

```
=====
남은 타일 || □: 7 ■: 8
=====

==컴퓨터 턴==
<1: ■0> [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 턴==
[1: □4] [2: □6] <3: ■8> [4: ■11]
플레이어의 차례입니다.
가져오고 싶은 타일의 색을 고르세요. (흑:0, 백:1):
```

- 플레이어는 가져오고 싶은 타일의 색을 고른다. 만일 남아있는 타일의 색이 하나라면 자동으로 가져오게 된다. 남아있는 타일이 없다면 바로 추리를 진행한다.

```
가져오고 싶은 타일의 색을 고르세요. (흑:0, 백:1):
0
가져온 타일은 흑색 타일 5번입니다.

=====
남은 타일 || □: 6 ■: 8
=====

==컴퓨터 턴==
<1: ■0> [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 턴==
[1: □4] [2: □5] [3: □6] <4: ■8> [5: ■11]
플레이어가 추리를 시작합니다.
추리하고싶은 위치와 숫자를 입력해주세요.
```

- 어떤 타일을 가져왔는지 출력한다.
- 타일이 추가된 게임판을 새로 출력한다.
- 플레이어가 추리를 진행한다. 컴퓨터 턴에서 추리하고 싶은 위치와 숫자를 입력한다.

```

=====
남은 타일 || □: 6 ■: 8
=====

==컴퓨터 턴==
<1: ■0> [2: □??] [3: □??] [4: ■??] [5: □??]

==플레이어 턴==
[1: □4] [2: □5] [3: □6] <4: ■8> [5: ■11]
플레이어가 추리를 시작합니다.
추리고싶은 위치와 숫자를 입력해주세요.
5 11
플레이어가 5번째 위치의 숫자가 11라고 추리합니다.
정답입니다!
컴퓨터의 흑색 타일 11번이 공개됩니다.

=====
남은 타일 || □: 6 ■: 8
=====

==컴퓨터 턴==
<1: ■0> [2: □??] [3: □??] [4: ■??] <5: □11>

==플레이어 턴==
[1: □4] [2: □5] [3: □6] <4: ■8> [5: ■11]
플레이어가 추리를 시작합니다.
추리고싶은 위치와 숫자를 입력해주세요.

```

- 5번째 위치가 11번인 것 같다고 추리한 모습이다.
- **플레이어가 정답을 맞힌 경우:** 컴퓨터의 해당 타일이 공개된다. 컴퓨터의 5번 위치에 있는 흑색 11번 타일이 []에서 <>로 변하며, 숫자도 ??에서 11로 변한 것을 알 수 있다. 이후 플레이어는 추리를 이어간다.

```

=====
남은 타일 || □: 6 ■: 8
=====

==컴퓨터 턴==
<1: ■0> [2: □??] [3: □??] [4: ■??] <5: □11>

==플레이어 턴==
[1: □4] [2: □5] [3: □6] <4: ■8> [5: ■11]
플레이어가 추리를 시작합니다.
추리고싶은 위치와 숫자를 입력해주세요.
4 11
플레이어가 4번째 위치의 숫자가 11라고 추리합니다.
틀렸습니다!
플레이어의 흑색 타일 5번이 공개됩니다.

=====
남은 타일 || □: 6 ■: 8
=====

==컴퓨터 턴==
<1: ■0> [2: □??] [3: □??] [4: ■??] <5: □11>

==플레이어 턴==
[1: □4] <2: □5> [3: □6] <4: ■8> [5: ■11]
컴퓨터의 차례입니다.

```

- **플레이어가 틀린 경우:** 4번째 위치가 11번인 것 같다고 잘못 추리한 모습이다. 플레이어가 해당 턴에 가져온 타일이 공개된다. 만일 남아있던 타일이 없어서 이번 턴에 가져온 타일이 없다면, 턴에 있는 타일 중 가장 낮은(왼쪽) 위치에 있으면서 아직 공개되지 않은 타일을 공개한다. 턴이 다시 컴퓨터에게로

넘어간다.

게임은 위와 같이 컴퓨터 - 플레이어 간의 턴으로 반복된다.

- 컴퓨터/플레이어의 추리가 정답/오답일 경우, 항상 게임이 끝났는지 (즉, 모든 타일이 공개된 선수가 있는지) 판단해야 한다.

```
=====
남은 타일 || □: 3 ■: 5
=====

==컴퓨터 턴==
<1: □0> <2: ■1> <3: ■2> <4: ■3> <5: ■4> <6: □6> [7: ■??] <8: □10>

==플레이어 턴==
<1: □1> [2: □2] <3: □3> [4: □4] <5: ■6> [6: □7] [7: ■9] <8: □11>
플레이어가 추리를 시작합니다.
추리하고싶은 위치와 숫자를 입력해주세요.
7 8
플레이어가 7번째 위치의 숫자가 8라고 추리합니다.
정답입니다!
컴퓨터의 백색 타일 8번이 공개됩니다.
플레이어의 승리입니다!

=====
남은 타일 || □: 3 ■: 5
=====

==컴퓨터 턴==
<1: □0> <2: ■1> <3: ■2> <4: ■3> <5: ■4> <6: □6> <7: ■8> <8: □10>

==플레이어 턴==
<1: □1> [2: □2] <3: □3> [4: □4] <5: ■6> [6: □7] [7: ■9] <8: □11>
게임이 종료되었습니다. 메뉴로 돌아갑니다.

[ 다빈치 코드 ]
=====
1. 게임 설명
2. 게임 시작
3. 종료
```

- **플레이어가 승리했을 경우:** 플레이어의 최종 추리가 정답이 되며 컴퓨터의 턴에 있는 모든 타일이 공개되었다. 이에 따라 플레이어의 승리라는 메시지가 출력된 후, 최종 게임 판이 출력된다. 게임이 종료되었다는 메시지와 함께 초기 메뉴판을 다시 출력한다.

```
컴퓨터가 추리를 시작합니다.
컴퓨터가 4번째 위치의 숫자가 2라고 추리합니다.
정답입니다!
플레이어의 백색 타일 2번이 공개됩니다.
컴퓨터의 승리입니다!

=====
남은 타일 || □: 0 ■: 0
=====

==컴퓨터 턴==
<1: □1> <2: □2> <3: ■3> [4: ■??] <5: ■5> [6: □??] <7: ■7> <8: ■9> <9: □10> [10: ■??] <11: □11> [12: ■??]

==플레이어 턴==
<1: □0> <2: ■0> <3: ■1> <4: ■2> <5: □3> <6: □4> <7: □5> <8: ■6> <9: □7> <10: □8> <11: ■8> <12: □9>
게임이 종료되었습니다. 메뉴로 돌아갑니다.
```

- **컴퓨터가 승리했을 경우:** 마찬가지로 승리메시지 출력 -> 최종 게임 판 출력 -> 초기 메뉴판 출력을 한다.

3. 기타 사항 (중요!)

- 남아있는 흑/백 타일, 컴퓨터 텍, 플레이어 텍은 반드시 링크드 리스트로 구현해야 한다.
- 링크드 리스트의 구현을 위해서라도, 각 타일을 나타내는 구조체를 정의하여 사용해야 한다.
→ 각 타일의 색 / 숫자 / 공개여부를 나타내야 하기 때문에 구조체를 사용하는 것이 좋다.
- 동적할당과 메모리 해제를 할 때는 malloc / calloc / realloc / free를 사용하지 않고, C++ new / delete를 사용하도록 한다.
→ 메모리 누수 (Memory Leak)이 없도록 해야 한다.
→ crtddb.h 헤더를 사용하여, _CrtDumpMemoryLeaks() 를 통해 디버그 모드에서 확인이 가능하다.
- 위 설명에서 '랜덤하게'라고 설명된 부분은 실제로 랜덤 함수를 사용해야 한다.
→ cstdlib 헤더를 사용하여, rand() 함수와 srand() 함수를 사용할 수 있다.
- 텍에서 타일을 오름차순 정렬할 때 sorting 알고리즘을 사용해야 할 텐데, linked list의 각 node를 물리적으로 움직이기 보다, 그저 값만 베껴 옮기는 방식으로 하면 구현이 더 편할 수도 있다.
- 게임판을 출력할 때 흑/백색, []/<> 괄호, ??/숫자를 나타낼 때는 cout 문에 삼항연산자를 쓰면 편할 수 있다.
→ cout << ... <<(visible?to_string(number):"??") << ...
- 이번 프로그래밍 과제의 코드 파일 구조는 자유입니다. 하나의 main.cpp 파일로 제출해도 되고, 필요에 따라 여러 .h 파일 및 .cpp 파일을 정의해도 좋습니다.